# Discrete Fourier Transform

The discrete Fourier transform (DFT) is described in detail in the book <cite>DAFX: Digital Audio Effects (second edition)</cite> in chapter 1.3.2 - Spectrum analysis of digital signals.

The DFT of a time discrete signal $x(n)$ is evaluated over a block of $N$ samples by the following equation:

$X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-j2\pi nk/K}, 0 \leq n < N, K \geq N$

For simplicity, we will first assume, that the blocklength $N$ is equal to the transform lenght $K$: $N = K$ which corresponds to no zero padding.

The inverse algorithm is the IDFT (inverse discrete Fourier transform):

$x(n) = \frac{1}{K} \sum_{k=0}^{K-1} X(k) \cdot e^{j2\pi nk/K}$.

The implementation of the DFT is usually called Fast Fourier Transform (FFT) and is part of most signal processing libraries. In the following, the equivalence between the DFT and the numpy version of the FFT is shown:

[1]

```
import numpy as np
import matplotlib.pyplot as plt
import sys
sys.path.insert(1, '../Basics')
import WaveFileHandling

# define parameters and random input signal
N = 2**6
K = N
x = np.random.randn(N)

# test the DFT implementation
X = np.zeros(x.shape, dtype = complex)
for k in range(K):
    for n in range(N):
        X[k] += x[n] * np.exp(-1j*2*np.pi*n*k/K)
assert np.sum(np.abs(X - np.fft.fft(x))) < 1e-6, 'DFT and FFT are not equal'
# test the IDFT implementation
y = np.zeros((N), dtype = complex)
for n in range(N):
    for k in range(K):
        y[n] += X[k] * np.exp(1j*2*np.pi*n*k/K) / K
assert np.sum(np.abs(x - y)) < 1e-6, 'IDFT is not the inverse of the DFT'

print('All tests passed successfully')
```

```
All tests passed successfully
```

## Time resolution and frequency resolution

If the time resolution (or sampling time) $T = \frac{1}{r}$ of the time discrete input data $x(n)$ is known, the frequency resolution $\Delta f$ of the DFT can be determined by the transform length $K$:

$\Delta f = \frac{1}{K \cdot T}$

The physical timestamp $t$ and the time discrete index $n$ has the following correspondency:

$t = n \cdot T$

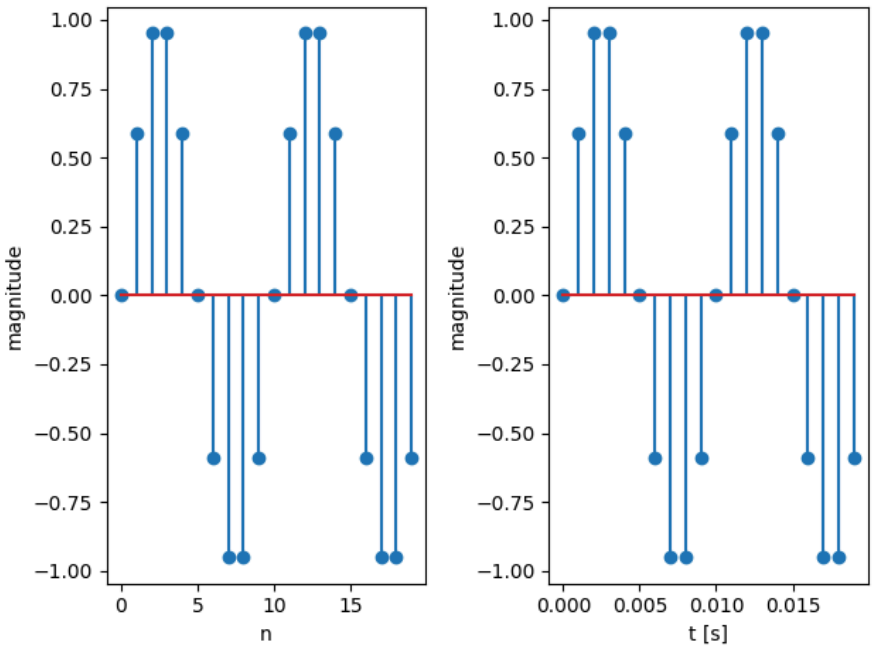
The physical frequency and the frequency discrete index $k$ has the following correspondency:

$f = k \cdot \Delta f = \frac{k}{K \cdot T}$

The relationship between the physical values (time and frequency) and the digital valus ($n$ and $k$) will be visualized on the x-axis for the example of a sinus with $100$ Hz and a sampling rate of $r = \frac{1}{T} = 1000$ Hz:

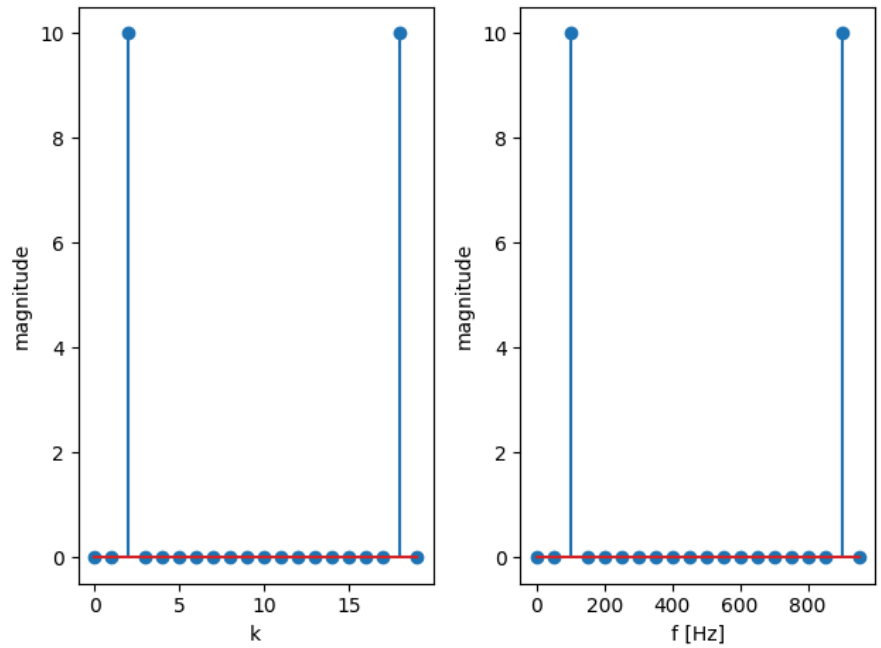
[2]

```
N = 20
r = 1000 # sampling rate in Hertz
T = 1/r # sampling period in seconds
FrequencyInHertz = 100

n_axis = np.arange(N)
PhysicalTimestamps = n_axis * T
SignalTimeDomain = np.sin(2*np.pi*FrequencyInHertz*PhysicalTimestamps)

plt.subplot(1,2,1)
plt.stem(n_axis, SignalTimeDomain)
plt.xlabel('n')
plt.ylabel('magnitude')
plt.subplot(1,2,2)
plt.stem(PhysicalTimestamps, SignalTimeDomain)
plt.xlabel('t [s]')
plt.ylabel('magnitude')

plt.tight_layout()
plt.show()
```




[3]

```
K = N

SignalFrequencyDomain = np.fft.fft(SignalTimeDomain, K)

Delta_f = 1 / (T*K)
k_axis = np.arange(K)
PhysicalFrequencies = k_axis * Delta_f

plt.subplot(1,2,1)
plt.stem(k_axis, np.abs(SignalFrequencyDomain))
plt.xlabel('k')
plt.ylabel('magnitude')
plt.subplot(1,2,2)
plt.stem(PhysicalFrequencies, np.abs(SignalFrequencyDomain))
plt.xlabel('f [Hz]')
plt.ylabel('magnitude')

plt.tight_layout()
plt.show()
```

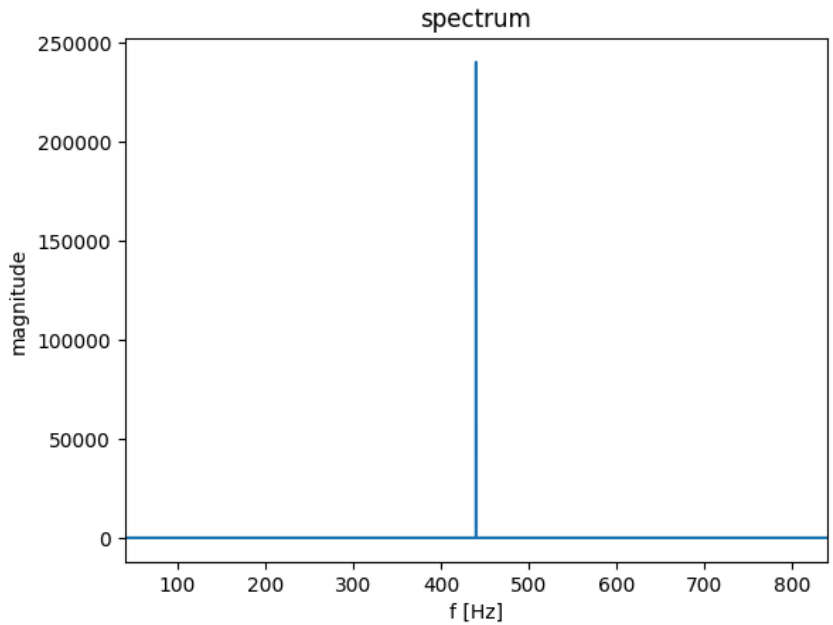



## Auralisation of DFT

In the following a simple signal is assumed, a monofrequent sinus. The corresponding DFT result is shown: the spectrum. Additionally, the signal is stored as a wave file for listening experiments, if the defined frequency is correctly applied to the wave file.

[4]

```
r = 48000
Frequency = 440 # frequency in Hertz
DurationInSeconds = 10
t = np.arange(DurationInSeconds*r)/r
x = np.sin(2*np.pi*Frequency*t)
K = x.shape[0]
Delta_f = r / K
X = np.abs(np.fft.fft(x, K))
f = np.arange(X.shape[0]) * Delta_f
plt.plot(f, X)
plt.xlim(Frequency-400, Frequency+400)
plt.xlabel('f [Hz]')
plt.ylabel('magnitude')
plt.title('spectrum')
plt.show()

WaveFileHandling.WriteWaveAsNumpyArray(x, r, 'sinus.wav')
```




## Zero padding

The DFT equation was already introduced above:

$X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-j2\pi nk/K}$

with $0 \leq n < N$ and $0 \leq k < K$. $N$ is the length of the time signal which is analysed and $K$ is the so-called transform length.

The frequency resolution of the DFT is evaluated by

$\Delta f = \frac{1}{KT}$ with $T$ equals to the sampling duration or the inverse of the sampling rate $r$.

The frequency resolution can be increased by increasing $K$. If $K > N$, the DFT fills the signal $x(n)$ with $K - N$ zeros at the end before the transformation to $X(k)$ is applied. This is called zero-padding. The increased frequency resolution gives no additional information about the spectrum. Instead the underlying continuous spectrum is sampled with a finer frequency resolution.

Zero padding has two motivations: Firstly, with zero padding it is possible to analyze a given frequency. Secondly, it is possible to use a transform length $K$, which is a power of $2$. Generally, this speed up the evaluation of the DFT, especially for low budget processors.

In order to show the effect of zero padding, a triangle signal is assumed in the following (plotted in black in the left plot):

$$x(t) = \begin{cases} 1 - |t|\,, & \text{if } -1 \leq t \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

The corresponding continuous spectrum can be evaluated by the Fourier transform:

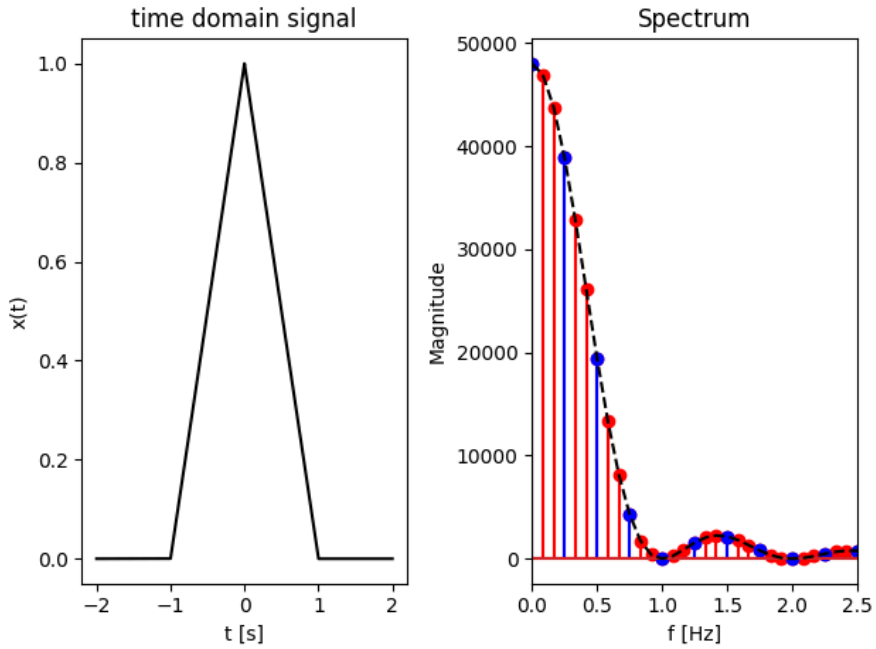
$X(f) = \int_{-\infty}^{\infty} x(t) \cdot e^{-j2\pi ft} dt = \text{sinc}^2(f)$

It is plotted as a dashed black line in the right plot.

The result of the DFT without zero padding is plotted in blue. The result of the DFT with zero padding with $K = 3 \cdot N$ is plotted in red. Due to the factor of $\frac{K}{N} = 3$, three times more frequencies are evaluated by the DFT. These additional frequencies has no more information, because the underlying true continuous spectrum (dashed black line) is only sampled more precisely.

[5]

```
r = 48000
T = 1/r
t = np.arange(4*r)/(r) - 2 # four seconds
x = np.maximum(1 - np.abs(t), 0.0)

def EvaluateSpectrum(x, K, r):
    X = np.fft.fft(x, K)
    NyquistIndex = int(K/2+1)
    X = np.abs(X[0:NyquistIndex])
    Deltaf = r/K
    f = np.arange(NyquistIndex) * Deltaf
    return X, f

N = x.shape[0]
X, f = EvaluateSpectrum(x, N, r)

K = 3*N
X_zeropadded, f_zeropadded = EvaluateSpectrum(x, K, r)

plt.subplot(1,2,1)
plt.plot(t, x, 'k')
plt.xlabel('t [s]')
plt.ylabel('x(t)')
plt.title('time domain signal')

plt.subplot(1,2,2)
plt.stem(f_zeropadded, X_zeropadded, 'r')
plt.stem(f, X, 'b')
plt.plot(f_zeropadded, (np.sinc(f_zeropadded)**2)*X[0], '--k') # the true spectrum
plt.xlim(0, 2.5)
plt.xlabel('f [Hz]')
plt.ylabel('Magnitude')
plt.title('Spectrum')

plt.tight_layout()
plt.show()
```




## Symmetry

If the input data is real valued, the output data is symmetric to the half of the sampling frequency:

$$X(k) = X^*(K-k)$$

For reducing the memory consumption, the second half symmetric part of the spectrum is usually dropped for further analysis, because it is redundant. The highest index, which has to be kept for analysis is the index of the Nyquist-frequency: $f_{\text{Nyquist}} = \frac{r}{2}$.

The zero-th index corresponds to the frequency $f = 0$ Hz, which is nothing else than the DC-part of the signal. For even values of $K$, the element at the Nyquist-frequency is the real part of the underlying spectrum, due to the symmetry-condition at this point:

$$X\left(k_{\text{Nyquist}}\right) = X^*\left(k_{\text{Nyquist}}\right)$$

[6]

```
N = np.random.randint(1000)
K = int(2**np.ceil(np.log2(N)))

x = np.random.randn(N)
X = np.fft.fft(x, K)

for k in range(1, K):
    assert np.abs(X[k] - np.conj(X[K-k])) < 1e-10, 'error in DFT symmetry'

NyquistIndex = K // 2 + 1
X_FirstHalfOfSpectrum = X[0:NyquistIndex]
Y = np.zeros((K), dtype = complex)
for k in range(K):
    if k < X_FirstHalfOfSpectrum.shape[0]:
        Y[k] = X_FirstHalfOfSpectrum[k]
    else:
        Y[k] = np.conj(Y[K- k])
assert np.sum(np.abs(X - Y)) < 1e-6, 'wrong reconstruction of second symmetric part
of the spectrum'
```

## Parsevals theorem

The Parseval theorem for continuous signals states, that the energy can be evaluated in time domain $x(t)$ or in frequency domain $X(f)$:

$$E = \int_{-\infty}^{\infty} x(t)dt = \int_{-\infty}^{\infty} X(f)df$$

This is also true for the DFT, with the exception of a scaling factor $\frac{1}{K}$:

$$E = \sum_{n=0}^{N-1} |x(n)|^2 = \frac{1}{K}\sum_{k=0}^{K-1} |X(k)|^2$$

In the case of even values of $K$ the Parseval theorem for the first part of the spectrum cannot be fullfilled exactly: For the Nyquist frequency, the imaginary part is lost. Therefore, the Parseval-theorem is not fullfilled exactly in the case of even values of $K$.

[7]

```
N = np.random.randint(1000)
K = N + np.random.randint(1000)

x = np.random.randn(N)
X = np.fft.fft(x, K)

# Parsevals theorem for full spectrum
E_x = np.sum(np.abs(x)**2)
E_X = np.sum(np.abs(X)**2) / K
assert np.abs(E_x - E_X)< 1e-10, 'error in evaluation of Parsevals theorem'

# Parseval theorem for the relevant first half spectrum
NyquistIndex = K // 2 + 1
X_FirstHalfOfSpectrum = X[0:NyquistIndex]
X2_FirstHalfOfSpectrum = np.abs(X_FirstHalfOfSpectrum)**2
X2_FirstHalfOfSpectrum[1:] *= 2
E_X = np.sum(X2_FirstHalfOfSpectrum) / K
if (K % 2) == 1:
    assert np.abs(E_X - E_x) < 1e-10, 'error in evaluation of Parseval theorem for
first half of spectrum'
```

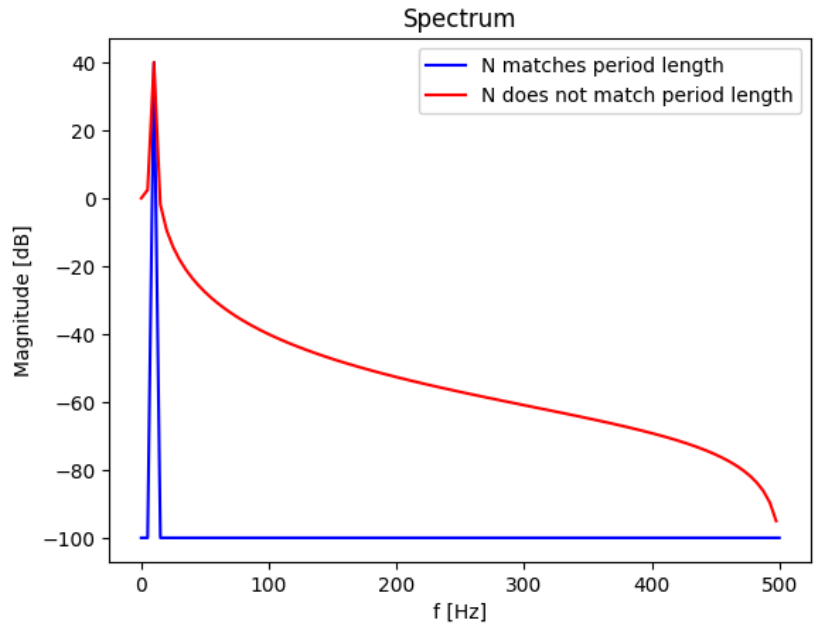
## Periodicity

The DFT or FFT assumes a periodical signal. The assumption of periodicity of the input signal lead to problems, if the signal to be analysed does not correspond to a whole period of the signal, i.e. $N$ is not an

integer multiple of the period length. In these cases, the beginning and ending of the block of length $N$ are not matching. This behaviour is visualized by two signals $x(n)$ and $y(n)$: $x(n)$ contains a whole period (or an integer number of periods) and $y(n)$ does not contain a whole period. The spectrum of $y(n)$ is plotted in red and contains a large amount of signal energy beside the maximum at 10 Hz. This large amount of energy beside the maximum is called frequency smearing or leakage of the DFT.

[8]

```
def EvaluateSpectrum(x, Fs):
    X = np.fft.fft(x)
    NyquistIndex = int(X.shape[0]/2+1)
    X = np.abs(X[0:NyquistIndex])
    f = np.arange(X.shape[0])/x.shape[0]*Fs
    X_level = 20*np.log10(X + 1e-5)
    return X_level, f

Fs = 1000
f = 10
n = np.arange(int(2*Fs/f))
x = np.cos(2*np.pi*f*n/Fs)
n = np.arange(int(2*Fs/f+1))
y = np.cos(2*np.pi*f*n/Fs)

X, f_x = EvaluateSpectrum(x, Fs)
Y, f_y = EvaluateSpectrum(y, Fs)

plt.plot(f_x, X, 'b')
plt.plot(f_y, Y, 'r')
plt.xlabel('f [Hz]')
plt.ylabel('Magnitude [dB]')
plt.title('Spectrum')
plt.legend(['N matches period length', 'N does not match period length'])
plt.show()
```




## Window function

One workaround for the conditon of the DFT of assuming periodic input signals is to use a window function $w(n)$ which is element-wise multiplied with the signal to be analysed $x(n)$. These window functions realises a fade in and fade out of the signal $x(n)$. A commonly used window function is the Hann window:

$$w(n) = \frac{1}{2}\left(1 - \cos\left(2\pi\frac{n+0.5}{N}\right)\right)$$

There exists a big bunch of different window functions. Most of them has a unique maximum in the middle and they are fading in and fading out to the left and right ending of the window function.

Therefore, in frequency domain these window functions are lowpasses.

In the following, the effect of the window function is shown for the DFT:

The upper plot is the time domain. The non-periodic signal $y(n)$ is plotted in red. The result of the multiplication with the Hann window is plotted in black:

$$y_{\text{windowed}}(n) = y(n) \cdot w(n)$$

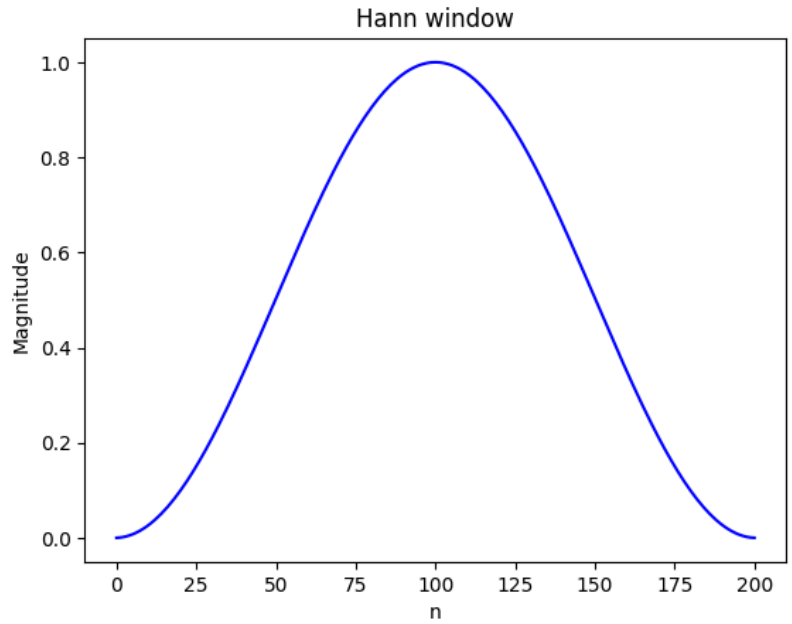
The upper plot shows the fading in and out due to the multiplication with the Hann window.

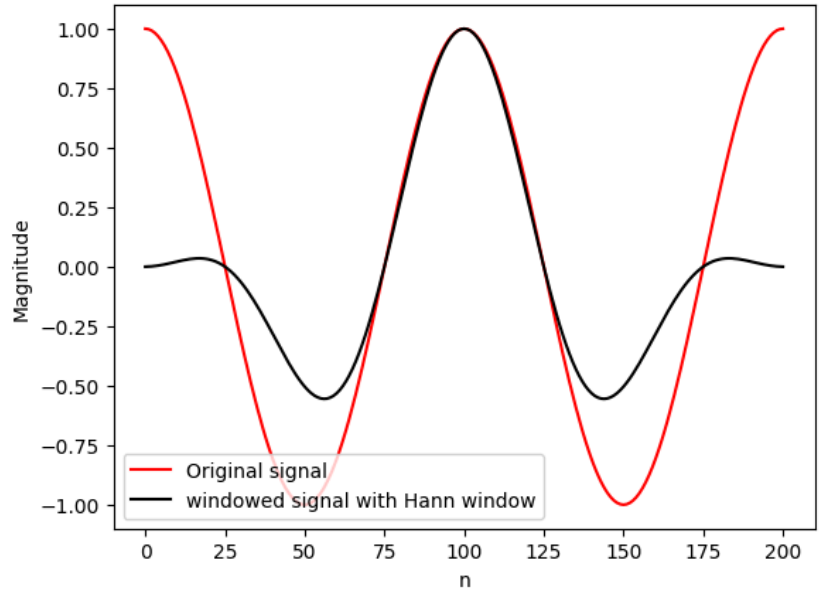
the lower plot shows the DFT of the periodic signal $x(b)$ in blue as a reference. The DFT of the non-periodic signal $y(n)$ is plotted again in red. The DFT of the windowed signal $y_{\text{windowed}}(n)$.

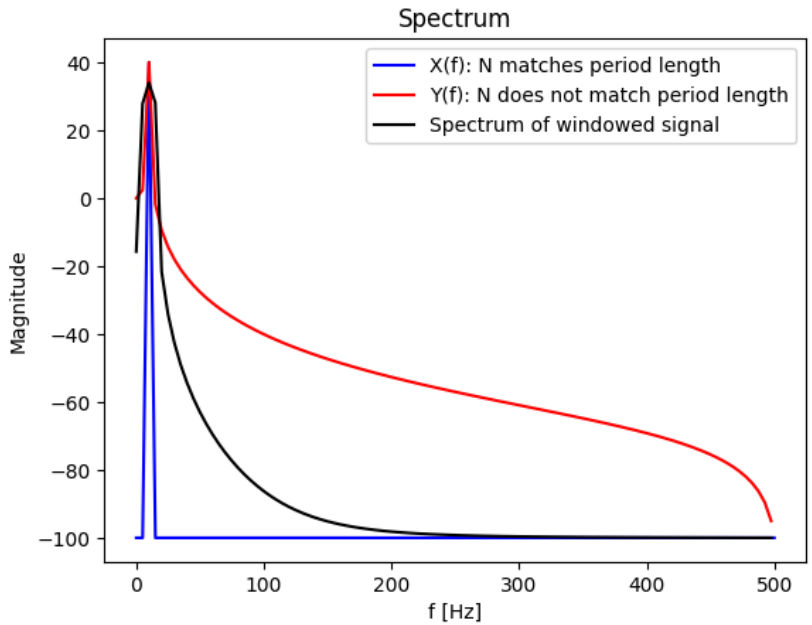
In lower plot shows the influence of the multiplication with the Hann window in frequency domain: The smearing is reduced properly.

[9]

```
def HannWindow(N):
    n = np.arange(N)
    return 0.5 * (1 - np.cos(2*np.pi*(n+0.5)/N))

w = HannWindow(y.shape[0])
y_windowed = y * w

plt.plot(n, w, 'b')
plt.title('Hann window')
plt.xlabel('n')
plt.ylabel('Magnitude')
plt.show()

plt.plot(n, y, 'r')
plt.plot(n, y_windowed, 'k')
plt.xlabel('n')
plt.ylabel('Magnitude')
plt.legend(['Original signal', 'windowed signal with Hann window'])
plt.show()

Y_windowed, f_y = EvaluateSpectrum(y_windowed, Fs)

plt.plot(f_x, X, 'b')
plt.plot(f_y, Y, 'r')
plt.plot(f_y, Y_windowed, 'k')
plt.xlabel('f [Hz]')
plt.ylabel('Magnitude')
plt.title('Spectrum')
plt.legend(['X(f): N matches period length', 'Y(f): N does not match period length',
'Spectrum of windowed signal'])
plt.show()
```






## Programming exercise:

The DFT algorithm can be described by a matrix multiplication:

$X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-j2\pi nk/K}, 0 \leq n < N$

$\rightarrow X = T_{\text{DFT}} \cdot x$

Evaluate the transform matrix $T_{\text{DFT}}$ in the procedure DFTByMatrixMultiplication.

Implement the option of zero padding ($K > N$) in the procedure DFTByMatrixMultiplication.

The multiplication with the window function HannWindow(N) can also explained by a matrix multiplication. Implement the corresponding procedure WindowingByMatrixMultiplication.

Both operations windowing and DFT can be combined in a single matrix multiplication.

[12]

```
def DFTByMatrixMultiplication(N, K = None):  # Erzeugt die DFT-Transformationsmatrix
T mit Form (K x N)
    if K is None:  # Falls kein K gegeben ist, nutzen wir den Standardfall ohne
Zero-Padding
        K = N  # Setzt Transformationslaenge gleich Signallaenge
    assert K ≥ N, 'K must be greater than or equal to N for zero padding'  # Zero-
Padding ist nur sinnvoll/freigegeben fuer K ≥ N

    n = np.arange(N)  # Zeitindex-Vektor n = [0, 1, ..., N-1]
    k = np.arange(K)[:, np.newaxis]  # Frequenzindex-Vektor k als Spaltenvektor fuer
Broadcasting
    T = np.exp(-1j * 2 * np.pi * k * n / K)  # Matrixelemente T[k,n] = exp(-
j*2*pi*n*k/K)
    return T  # Gibt die fertige DFT-Matrix zurueck

def WindowingByMatrixMultiplication(N):  # Erzeugt die Matrix fuer die Fensterung
als lineare Abbildung
    w = HannWindow(N)  # Berechnet die N Hann-Fensterkoeffizienten
    T = np.diag(w)  # Baut eine Diagonalmatrix, sodass T @ x genau x * w ergibt
    return T  # Gibt die Fensterungsmatrix zurueck

def WindowingAndDFTByMatrixMultiplication(N, K = None):  # Kombiniert Fensterung und
DFT in einer einzigen Matrix
    T_dft = DFTByMatrixMultiplication(N, K)  # DFT-Matrix (mit optionalem Zero-
Padding) erzeugen
    T_window = WindowingByMatrixMultiplication(N)  # Fensterungsmatrix erzeugen
    T = np.matmul(T_dft, T_window)  # Kombinierte Abbildung: erst fenstern, dann DFT
    return T  # Gibt die kombinierte Matrix zurueck

import unittest

class TestProgrammingExercise(unittest.TestCase):

    def test_DFTByMatrixMultiplication(self):
        N = np.random.randint(1000) + 1
        x = np.random.randn(N)
        T = DFTByMatrixMultiplication(N)
        y1 = np.matmul(T, x)
        y2 = np.fft.fft(x)
        self.assertGreater(10*np.log10(np.sum(np.abs(y2)**2)/np.sum(np.abs(y1-
y2)**2+1e-16)), 100)

    def test_DFTByMatrixMultiplicationZeroPadding(self):
        N = np.random.randint(1000) + 1
        K = int(2**np.ceil(np.log2(N)))
        x = np.random.randn(N)
        T = DFTByMatrixMultiplication(N, K)
        y1 = np.matmul(T, x)
        y2 = np.fft.fft(x, n = K)
        self.assertGreater(10*np.log10(np.sum(np.abs(y2)**2)/np.sum(np.abs(y1-
y2)**2+1e-16)), 100)

    def test_WindowingByMatrixMultiplication(self):
        N = np.random.randint(1000) + 1
        K = N + np.random.randint(10)
        x = np.random.randn(N)
        w = HannWindow(N)
        T = WindowingByMatrixMultiplication(N)
        y1 = np.matmul(T, x)
        y2 = x*w
        self.assertGreater(10*np.log10(np.sum(np.abs(y2)**2)/np.sum(np.abs(y1-
y2)**2+1e-16)), 100)

    def test_WindowingAndDFTByMatrixMultiplication(self):
        N = np.random.randint(1000) + 1
        K = N + np.random.randint(10)
        x = np.random.randn(N)
        w = HannWindow(N)
```

```
        T = WindowingAndDFTByMatrixMultiplication(N)
        y1 = np.matmul(T, x)
        y2 = np.fft.fft(x*w)
        self.assertGreater(10*np.log10(np.sum(np.abs(y2)**2)/np.sum(np.abs(y1-
y2)**2+1e-16)), 100)

unittest.main(argv=['''], verbosity=2, exit=False)
```

```
Collapse
test_DFTByMatrixMultiplication
(__main__.TestProgrammingExercise.test_DFTByMatrixMultiplication) ... ok
test_DFTByMatrixMultiplicationZeroPadding
(__main__.TestProgrammingExercise.test_DFTByMatrixMultiplicationZeroPadding) ... ok
test_WindowingAndDFTByMatrixMultiplication
(__main__.TestProgrammingExercise.test_WindowingAndDFTByMatrixMultiplication) ... ok
test_WindowingByMatrixMultiplication
(__main__.TestProgrammingExercise.test_WindowingByMatrixMultiplication) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.110s

OK
```

```
<unittest.main.TestProgram at 0x26aa1b8fb10>
```

## Exam preparation

Prove the symmetry of the DFT: $X(k) = X^*(K-k)$.

A sinus with $f = 440$ Hz and amplitude $\hat{x} = 1$ is sampled with a sampling rate of $r = 1$ kHz. Evaluate the first three samples $x(n)$. Evaluate the result of the DFT over the first three samples with $N = K = 3$. Evaluate the energy in the time domain and in the frequency domain.

A signal has a sampling rate of $r = 48$ kHz. The DFT uses a block length of $N = 1000$ and a transform length of $K = 1024$. Evaluate the frequency resolution $\Delta f$. Is zero-padding used? Which index $k$ corresponds to $f = 50$ Hz? How many indices can be dropped after transform due to the symmetry?

Give a list of advantages and disadvantages of the DFT.

The z-Transform can be used to evaluate the Fourier transform of a time discrete signal by the following replacement: $z \leftarrow e^{j2\pi f/r}$: $X(f) = \sum_{n=-\infty}^{\infty} x(n) \cdot e^{-j2\pi f n/r}$. Under which conditions is the output of the DFT $X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-j2\pi nk/N}$ identical to the Fourier transform of a time discrete signal?

## Exam preparation - Lösungen

### 1) Symmetrie der DFT beweisen

Für ein reelles Signal $x(n) \in \mathbb{R}$ gilt:

$$X(k) = \sum_{n=0}^{K-1} x(n) e^{-j2\pi nk/K}$$

Dann ist

$$X^*(K-k) = \left( \sum_{n=0}^{K-1} x(n) e^{-j2\pi n(K-k)/K} \right)^*$$

$$= \sum_{n=0}^{K-1} x(n) e^{+j2\pi n(K-k)/K} = \sum_{n=0}^{K-1} x(n) e^{j2\pi n} e^{-j2\pi nk/K}$$

Da $e^{j2\pi n} = 1$ für ganzzahliges $n$:

$$X^*(K-k) = \sum_{n=0}^{K-1} x(n) e^{-j2\pi nk/K} = X(k)$$

Also gilt für reelle Signale:

$$\boxed{X(k) = X^*(K-k)}$$

### 2) Sinus: $f = 440$ Hz, $\hat{x} = 1$, $r = 1000$ Hz, $N = K = 3$

Signal:

$$x(n) = \sin\left(2\pi \frac{440}{1000} n\right) = \sin(0.88\pi n)$$

Erste drei Samples:

$x(0) = 0$
$x(1) \approx 0.3681246$
$x(2) \approx -0.6845471$

DFT mit $K = 3$:

$$X(k) = \sum_{n=0}^{2} x(n) e^{-j2\pi nk/3}$$

Ergebnis:

$X(0) \approx -0.3164226$
$X(1) \approx 0.1582113 - j\,0.9116404$
$X(2) \approx 0.1582113 + j\,0.9116404$

Energie im Zeitbereich:

$$E_t = \sum_{n=0}^{2} |x(n)|^2 \approx 0.60412$$

Energie im Frequenzbereich (Parseval):

$$E_f = \frac{1}{K}\sum_{k=0}^{2}|X(k)|^2 = \frac{1}{3}\sum_{k=0}^{2}|X(k)|^2 \approx 0.60412$$

Damit ist $E_t \approx E_f$.

### 3) $r = 48$ kHz, $N = 1000$, $K = 1024$

Frequenzauflösung:

$$\Delta f = \frac{r}{K} = \frac{48000}{1024} = 46.875\,\text{Hz}$$

Zero Padding: Ja, weil $K > N$ (es werden 24 Nullen angehängt).
Index für $f = 50$ Hz:

$$k = \frac{f}{\Delta f} = \frac{50}{46.875} \approx 1.067$$

Also liegt 50 Hz zwischen Bin $k = 1$ und $k = 2$; nächster Bin ist $k = 1$.

Anzahl verwerfbarer Indizes wegen Symmetrie (reelles Signal): Behalten wird bis Nyquist: $K/2 + 1 = 513$ Bins. Verwerfbar: $1024 - 513 = 511$ Bins.

### 4) Vorteile und Nachteile der DFT

Vorteile:

- Spektralanalyse diskreter Signale klar und standardisiert.
- Mit FFT sehr effizient berechenbar.
- Parseval-Zusammenhang für Energie nutzbar.
- Gut kombinierbar mit Fensterung und Zero Padding.

Nachteile:

- Leckeffekte bei nicht periodenkonsistenten Signalsegmenten.
- Begrenzte Frequenzauflösung durch $\Delta f = r/K$.
- Frequenzen zwischen Bins werden nur angenähert dargestellt.
- DFT ohne FFT ist rechenintensiv.

### 5) Wann ist DFT identisch zur Fourier-Transformierten?

Die DFT-Werte sind Abtastungen der DTFT bei

$$\omega_k = \frac{2\pi k}{N} \quad \Leftrightarrow \quad f_k = \frac{k\,r}{N}$$

Identisch zur Fourier-Transformierten des betrachteten zeitdiskreten Signals ist die DFT, wenn:

- das Signal im betrachteten Intervall $0 \ldots N-1$ vollständig enthalten ist (außerhalb davon 0), oder
- das Signal $N$-periodisch ist und periodenkonsistent analysiert wird.

Andernfalls analysiert die DFT die periodisch fortgesetzte/gefensterte Version des Signals, wodurch z. B. Leckeffekte entstehen können.

## Summary

After working with this Jupyter Notebook, you should be able to answer the following questions:

- What is the difference between the Fourier transform, the Discrete Fourier transform and the Fast Fourier transform?
- What is the meaning of the discrete indices $n$ and $k$ and the relationship to the physical values time $t$ and frequency $f$?
- What is the motivation for zero padding?
- What is the effect of zero padding on the result of the DFT $X(k)$?
- Why is the second half of the spectrum $X(k)$ typically ignored for signal processing algorithms?
- What is the motivation for using a window function, e.g. the Hann window?

## Summary - verständlich erklärt

Nach diesem Notebook solltest du die folgenden Punkte sicher erklären können:

### 1) Unterschied zwischen Fourier-Transform, DFT und FFT

- **Fourier-Transform (FT):** arbeitet mit kontinuierlicher Zeit bzw. kontinuierlicher Frequenz. Sie beschreibt das ideale mathematische Spektrum eines Signals.
- **Discrete Fourier Transform (DFT):** arbeitet mit endlich vielen diskreten Samples. Sie liefert Spektralwerte nur an diskreten Frequenzstellen (Bins).
- **Fast Fourier Transform (FFT):** ist **kein anderes Ergebnis**, sondern nur ein sehr effizienter Algorithmus zur Berechnung der DFT.

Merksatz: **FFT berechnet DFT schnell.**

### 2) Bedeutung von $n$ und $k$ sowie Bezug zu $t$ und $f$

- $n$ ist der **Zeitindex** im Sample-Bereich.
- $k$ ist der **Frequenzindex** im Spektrum.

Mit Abtastrate $r$ und Abtastdauer $T = 1/r$ gilt:

$$t = nT$$

Und für die Frequenzachse der DFT mit Transformationslänge $K$:

$$\Delta f = \frac{1}{K\,T} = \frac{r}{K}, \quad f = k\,\Delta f$$

Das hast du im Notebook in den Plot-Beispielen mit Achsen in $n/t$ und $k/f$ gesehen.

## 3) Motivation für Zero Padding

Zero Padding bedeutet: wenn $K > N$, werden hinten Nullen an das Signal angehängt.

Warum macht man das?

- feinere Frequenzabtastung im Spektrum ($\Delta f$ wird kleiner),
- bestimmte Frequenzen können genauer getroffen werden,
- oft günstig für FFT-Implementierungen (z. B. $K$ als Zweierpotenz).

## 4) Effekt von Zero Padding auf $X(k)$

- Das Spektrum wird **dichter abgetastet** (mehr Bins).
- Die Kurve wirkt glatter bzw. genauer interpoliert.
- **Wichtig:** Es entsteht **keine neue physikalische Information** über das Signal.

Genau das zeigte das Notebook mit dem Vergleich ohne und mit Zero Padding beim Dreieckssignal.

## 5) Warum wird die zweite Spektrumshälfte oft ignoriert?

Bei **reellem** Eingangssignal gilt konjugiert-komplexe Symmetrie:

$$X(k) = X^*(K-k)$$

Damit steckt in der zweiten Hälfte keine neue Information, sie ist redundant. Deshalb nutzt man in der Praxis meist nur die erste Hälfte bis zur Nyquist-Frequenz.

## 6) Motivation für Fensterfunktionen (z. B. Hann)

Die DFT nimmt periodische Fortsetzung des betrachteten Blocks an. Wenn Blockanfang und Blockende nicht zusammenpassen, entstehen Sprünge und damit **Leckeffekte** (Smearing) im Spektrum.

Fensterung (z. B. Hann) macht ein sanftes Ein- und Ausblenden:

- reduziert Sprünge am Blockrand,
- reduziert Leckeffekte deutlich,
- macht Peaks im Spektrum oft besser interpretierbar.

Nachteil: auch Fensterung verändert das Spektrum (z. B. Amplituden-/Bandbreiten-Effekte), ist aber in vielen Anwendungen der bessere Kompromiss.